

# S2S-04: Step 4 — Prepare: Export and Pre-Stage

**Series:** S2S — SaaS to SaaS Migration | **Notebook:** 4 of 9 | **Phase:** Upgrade |  
**Step:** Prepare | **Created:** March 2026 | **Last Updated:** 09/24/2026

## Overview

With the target tenant architecture designed (Step 3), this step builds the foundation. Preparation is the longest step in any SaaS-to-SaaS migration — it involves exporting configuration from the source tenant, provisioning the target tenant, deploying IAM, installing ActiveGates, preparing Kubernetes operators, and freezing the source tenant to prevent drift during execution.

Every task in this notebook must be completed and validated before Step 5 (Execute) begins. Skipping preparation tasks is the primary cause of migration rollbacks.

## S2S Migration Framework

| Phase   | Steps                                         | Focus                                                  |
|---------|-----------------------------------------------|--------------------------------------------------------|
| Plan    | 1. Discover → 2. Strategize → 3. Design       | Understand what exists, choose approach, design target |
| Upgrade | <b>4. Prepare</b> → 5. Execute → 6. Integrate | Build target, move config, connect integrations        |
| Run     | 7. Expand → 8. Enable → 9. Optimize           | Roll out agents, enable teams, tune and decommission   |

**You are here: Step 4 — Prepare.** You have designed the target architecture (Step 3). Now you export configuration, provision the target tenant, and stage all prerequisites before execution begins.

---

## Table of Contents

---

1. [Target Tenant Provisioning](#)
  2. [SSO and IAM Setup](#)
  3. [Monaco Bulk Export](#)
  4. [ActiveGate Provisioning](#)
  5. [Kubernetes Operator Preparation](#)
  6. [Configuration Freeze](#)
  7. [Rollback Procedure](#)
  8. [Step Completion Checklist](#)
- 

## Prerequisites

---

| Requirement                      | Details                                                                                                       |
|----------------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>Step 3 Complete</b>           | Target tenant architecture designed — IAM, buckets, OpenPipeline, deployment order documented                 |
| <b>Source Tenant Access</b>      | Admin access with <code>ReadConfig</code> , <code>settings.read</code> , <code>activeGates.read</code> scopes |
| <b>Target Tenant Provisioned</b> | Dynatrace SaaS environment with Grail enabled                                                                 |
| <b>Monaco CLI</b>                | <code>monaco v2.x</code> installed and configured                                                             |
| <b>Terraform</b>                 | <code>terraform &gt;= 1.0</code> with Dynatrace provider <code>~&gt; 1.91</code>                              |
| <b>Helm</b>                      | <code>helm &gt;= 3.x</code> for Kubernetes operator deployment                                                |
| <b>Change Window</b>             | Approved change window for configuration freeze                                                               |

## Order of Operations

This notebook covers **operations 1–3** of the Upgrade phase:

| # | Operation                                            | Section      | Dependency                 |
|---|------------------------------------------------------|--------------|----------------------------|
| 1 | <b>Provision target tenant and configure SSO/IAM</b> | Sections 1–2 | Step 3 design deliverables |
| 2 | <b>Export configuration from source tenant</b>       | Section 3    | Source tenant access       |
| 3 | <b>Deploy ActiveGates and prepare K8s operators</b>  | Sections 4–5 | Target tenant provisioned  |
| 4 | Import configuration to target tenant                | Step 5       | Operations 1–3 complete    |
| 5 | Redirect agents to target tenant                     | Step 5       | Configuration imported     |
| 6 | Validate data flow in target tenant                  | Step 5       | Agents redirected          |
| 7 | Reconnect cloud integrations                         | Step 6       | Agents reporting           |
| 8 | Migrate dashboards, workflows, and extensions        | Step 6       | Integrations connected     |

**Bold** operations are covered in this notebook. Operations 4–8 are covered in Steps 5 and 6.

## 1. Target Tenant Provisioning

The target tenant must be provisioned and configured with baseline settings before any configuration is imported.

### Provisioning Checklist

| Item                          | Action                                                                          | Status                   |
|-------------------------------|---------------------------------------------------------------------------------|--------------------------|
| <b>Region selection</b>       | Choose region closest to primary workload (same region as source if relocation) | <input type="checkbox"/> |
| <b>Environment activation</b> | Activate SaaS environment via Dynatrace Account Management                      | <input type="checkbox"/> |
| <b>Grail enablement</b>       | Verify Grail is enabled (default for new SaaS tenants)                          | <input type="checkbox"/> |

| Item                         | Action                                                                                                                                 | Status                   |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|--------------------------|
| <b>API token creation</b>    | Create token with <code>writeConfig</code> , <code>readConfig</code> , <code>settings.write</code> , <code>settings.read</code> scopes | <input type="checkbox"/> |
| <b>OAuth client creation</b> | Create OAuth client for account-level API access (IAM management)                                                                      | <input type="checkbox"/> |
| <b>Cluster version</b>       | Verify target tenant is on same or newer cluster version than source                                                                   | <input type="checkbox"/> |

## Region Considerations

| Scenario                    | Region Strategy                                                  |
|-----------------------------|------------------------------------------------------------------|
| <b>Relocation</b>           | Same region as source (minimize latency change)                  |
| <b>Consolidation</b>        | Region closest to majority of monitored infrastructure           |
| <b>Cloud transformation</b> | Region matching the target cloud provider's primary region       |
| <b>Compliance-driven</b>    | Region that satisfies data residency requirements (EU, US, APAC) |

## Pre-Migration Baseline: Entity Counts

Before any migration begins, capture entity counts from the source tenant. These serve as validation targets after agent cutover in Step 5.

```
// Source tenant: baseline host count
fetch dt.entity.host
| summarize host_count = count()
| fieldsAdd entity_type = "Hosts"

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "HOST"
//   | summarize host_count = count()
//   | fieldsAdd entity_type = "Hosts"
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
// entities
// than the classic entity store; for a pre-migration discovery inventory keep
// the
// classic query above.
```

```
// Source tenant: baseline service count
fetch dt.entity.service
| summarize service_count = count()
| fieldsAdd entity_type = "Services"

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "SERVICE"
//   | summarize service_count = count()
//   | fieldsAdd entity_type = "Services"
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities
// than the classic entity store; for a pre-migration discovery inventory keep
the
// classic query above.
```

```
// Source tenant: baseline process group count
fetch dt.entity.process_group
| summarize pg_count = count()
| fieldsAdd entity_type = "Process Groups"

// Smartscape note (dt.entity.* is deprecated but still functional):
Smartscape models
// individual processes, not process GROUPS – smartscapeNodes "PROCESS" is a
different
// granularity (process instances), so its count is not comparable to a
process-group
// count. Keep the classic dt.entity.process_group query above.
```

```
// Source tenant: Grail bucket inventory for migration planning
fetch dt.system.buckets
| fields display_name, dt.system.table, retention_days,
estimated_uncompressed_bytes
| fieldsAdd size_gb = estimated_uncompressed_bytes / 1073741824.0
| sort size_gb desc
```

**Record these counts.** After agent cutover in Step 5, you will re-run these queries against the target tenant and compare. Entity counts within 5% indicate a successful migration. Counts below 90% indicate missing agents or misconfigured host groups.

## 2. SSO and IAM Setup

---

IAM is the first configuration deployed to the target tenant. Users need access to validate subsequent deployments, and IAM policies reference resources that will be created in later steps — but the group structure and base policies can be deployed now.

### SAML/SSO Configuration

| Step | Action                                                                                       | Notes                          |
|------|----------------------------------------------------------------------------------------------|--------------------------------|
| 1    | Navigate to <b>Account Management</b> → <b>Identity &amp; Access Management</b> → <b>SSO</b> | Target tenant                  |
| 2    | Configure SAML identity provider (same IdP as source if consolidation)                       | Copy metadata URL from IdP     |
| 3    | Map IdP groups to Dynatrace groups                                                           | Use the IAM design from Step 3 |
| 4    | Enable SSO enforcement (after initial admin access is confirmed)                             | Do not lock out admin accounts |
| 5    | Test SSO login with at least two different group memberships                                 | Verify policy inheritance      |

**Warning:** Do not enable SSO enforcement until you have confirmed that at least one admin account can log in via SSO. A misconfigured SSO with enforcement enabled will lock all users out of the tenant.

### Terraform IAM Deployment

IAM is **Terraform-only** — Monaco cannot manage account-level IAM. Deploy the group structure and base policies designed in Step 3:

```
# Initialize Terraform with Dynatrace provider
cd terraform/iam
terraform init

# Plan IAM deployment
terraform plan -var-file="target-tenant.tfvars"

# Apply IAM configuration
terraform apply -var-file="target-tenant.tfvars"
```

## IAM Deployment Order

| Order | Resource        | Terraform Resource Type                    |
|-------|-----------------|--------------------------------------------|
| 1     | Groups          | <code>dynatrace_iam_group</code>           |
| 2     | Policies        | <code>dynatrace_iam_policy</code>          |
| 3     | Policy bindings | <code>dynatrace_iam_policy_bindings</code> |

**Note:** For the initial deployment, create policies with broad `ALLOW` statements. After all configuration is imported (Step 5) and validated, tighten policies with `WHERE` clauses that reference specific buckets, schemas, and security contexts.

## 3. Monaco Bulk Export

Monaco `download` exports all configuration from the source tenant into a structured directory. This export becomes the input for `monaco deploy` in Step 5, or can be packaged for upload to the SaaS Upgrade Assistant (SUA) on the target tenant.

## Export Commands

```
# Set environment variables
export DT_SOURCE_URL="https://&lt;source-env-id&gt;.live.dynatrace.com"
export DT_SOURCE_TOKEN="dt0c01.xxx..."

# Full export of source tenant
monaco download \
  --environment-url "$DT_SOURCE_URL" \
  --token "$DT_SOURCE_TOKEN" \
  --output-folder ./export \
  --force

# Export specific configuration types only
monaco download \
  --environment-url "$DT_SOURCE_URL" \
  --token "$DT_SOURCE_TOKEN" \
  --output-folder ./export-settings \
  --only-settings \
  --force
```

## Automated Export with SUA Packaging

For a streamlined workflow that handles Monaco download, checksum verification, and SUA-compatible `.tar.gz` packaging in a single command, see **S2S-10: Migration Scripts**. The scripts automate:

- Platform-aware Monaco binary download (macOS, Linux, Windows)
- Temporary read-only API token creation with minimal scopes
- Full `monaco download` execution
- Packaging in SaaS Upgrade Assistant format ( `.tar.gz` with `exportMetadata.json` )

**Important:** The SaaS Upgrade Assistant requires `.tar.gz` format — `.zip` archives are rejected. If packaging manually, ensure the archive contains an `exportMetadata.json` at the root and the exported config in an `export/` subdirectory. See **S2S-10** for the exact format specification.

## Export Directory Structure

After `monaco download`, the export directory contains:

```

export/
├─ project/
│   ├─ alerting-profile/           # Alerting profiles
│   ├─ anomaly-detection-*/       # Anomaly detection rules
│   ├─ auto-tag/                  # Auto-tag rules
│   ├─ dashboard/                 # Classic dashboards
│   ├─ management-zone/          # Management zones
│   ├─ notification/              # Notification integrations
│   ├─ request-attributes/        # Request attributes
│   ├─ slo/                       # SLO definitions
│   ├─ synthetic-*/               # Synthetic monitors
│   └─ ... (50+ config types)
├─ manifest.yaml                  # Monaco manifest
└─ delete.yaml                    # Deletion tracking

```

## Export Validation

After export, validate the output before proceeding:

```

# Count exported configuration items
find ./export/project -name "*.json" -o -name "*.yaml" | wc -l

# Verify manifest is valid
monaco deploy --dry-run \
  --environment-url "$DT_TARGET_URL" \
  --token "$DT_TARGET_TOKEN" \
  --manifest ./export/manifest.yaml

# Check for entity ID references that need remapping
grep -r "HOST-\\|SERVICE-\\|PROCESS_GROUP-" ./export/project/ | wc -l

```

**Important:** Monaco download **cannot export** the following:

- Cloud provider credentials (AWS IAM roles, Azure service principals, GCP service account keys)
- Account-level IAM (groups, policies, bindings) — use Terraform
- ActiveGate tokens and connection info
- Synthetic private location credentials

These must be recreated manually in the target tenant.

## 4. ActiveGate Provisioning

ActiveGates in the source tenant cannot be "moved" to the target tenant. New ActiveGates must be deployed and connected to the target tenant. The source ActiveGates remain operational until cutover is validated.

### ActiveGate Sizing

| Role                   | CPU      | Memory | Disk  | Notes                         |
|------------------------|----------|--------|-------|-------------------------------|
| Environment ActiveGate | 2+ cores | 4+ GB  | 20 GB | Routing, API access           |
| Cluster ActiveGate     | 4+ cores | 8+ GB  | 50 GB | Extensions, synthetic         |
| Multi-purpose          | 4+ cores | 8+ GB  | 50 GB | Combined routing + extensions |

### Deployment Steps

| Step | Action                                                    | Validation                                         |
|------|-----------------------------------------------------------|----------------------------------------------------|
| 1    | Download ActiveGate installer from target tenant          | Verify installer matches target tenant ID          |
| 2    | Deploy to designated hosts (same hosts or new hosts)      | Check <code>oneagentctl --get-server</code> output |
| 3    | Assign ActiveGate groups (zones) matching source topology | Verify group assignment in target tenant UI        |
| 4    | Configure network zones if used                           | Match source network zone structure                |
| 5    | Verify connectivity from monitored hosts to new AGs       | Test TCP connectivity on port 443                  |

### ActiveGate Zone Assignment

If the source tenant uses ActiveGate groups (zones) to segment traffic, replicate the same zone structure in the target tenant:

| Source Zone     | Purpose              | Target Zone     | ActiveGate Count    |
|-----------------|----------------------|-----------------|---------------------|
| us-east-prod    | Production workloads | us-east-prod    | 2 (HA pair)         |
| us-east-nonprod | Dev/staging          | us-east-nonprod | 1                   |
| eu-west-prod    | EU production        | eu-west-prod    | 2 (HA pair)         |
| extensions      | Extension execution  | extensions      | 1 (host-based only) |

**Extensions 2.0 requirement:** Extensions 2.0 require a **host-based ActiveGate** — not a Kubernetes-based ActiveGate. Plan at least one host-based AG if extensions are in scope.

## Verify ActiveGate Connectivity

After deploying ActiveGates to the target tenant, verify they are reporting:

```
// Target tenant: verify ActiveGates are connected and reporting
// Note: ActiveGate entity type varies by deployment; host entities report AG
status
fetch dt.entity.host
| filter contains(entity.name, "activegate") or contains(entity.name,
"ActiveGate")
| fields entity.name, id, state
| sort entity.name asc

// Smartscape note (dt.entity.* is deprecated but still functional): this
query uses the
// classic-only field state, which has NO Smartscape node equivalent
(Smartscape expresses
// liveness via node lifetime, not a state field). Keep the classic query
above.
// Other fields do map: entity.name -> name.
```

## 5. Kubernetes Operator Preparation

For environments running Kubernetes, the Dynatrace Operator must be configured to report to the target tenant. This involves updating the DynaKube custom resource and Helm values.

## Preparation Steps

| Step | Action                                                 | Notes                                                                                                                                         |
|------|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Generate new API token and PaaS token in target tenant | Scopes: <code>activeGateTokenManagement.create</code> , <code>entities.read</code> , <code>DataExport</code> , <code>metrics.read</code>      |
| 2    | Create Kubernetes secret for target tenant             | <code>kubectl create secret generic dynakube-target --from-literal=apiToken=&lt;token&gt; --from-literal=dataIngestToken=&lt;token&gt;</code> |
| 3    | Prepare updated DynaKube CR with target tenant URL     | Do <b>not</b> apply yet — wait for Step 5                                                                                                     |
| 4    | Validate Helm chart version compatibility              | Target tenant cluster version must support the operator version                                                                               |

## DynaKube CR Template (Target Tenant)

```
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://<target-env-id>.live.dynatrace.com/api
  tokens: dynakube-target
  oneAgent:
    cloudNativeFullStack:
      tolerations:
        - effect: NoSchedule
          operator: Exists
  activeGate:
    capabilities:
      - routing
      - kubernetes-monitoring
      - dynatrace-api
  resources:
    requests:
      cpu: 500m
      memory: 512Mi
    limits:
      cpu: "1"
      memory: 1Gi
```

## Helm Chart Preparation

```
# Add Dynatrace Helm repo (if not already added)
helm repo add dynatrace https://raw.githubusercontent.com/Dynatrace/dynatrace-operator/main/config/helm/repos/stable
helm repo update

# Check available versions
helm search repo dynatrace/dynatrace-operator --versions

# Dry-run to validate (do NOT install yet)
helm upgrade --install dynatrace-operator dynatrace/dynatrace-operator \
  --namespace dynatrace \
  --create-namespace \
  --dry-run
```

**Do not apply the DynaKube CR yet.** Applying it now would cause agents to start reporting to the target tenant before configuration is imported. Wait for Step 5.

## 6. Configuration Freeze

A configuration freeze prevents drift between the exported configuration and the source tenant during the migration window. Changes made after the export but before import will be lost.

### Freeze Scope

| Category                  | Frozen? | Notes                                                 |
|---------------------------|---------|-------------------------------------------------------|
| Settings changes          | Yes     | No new alerting rules, detection rules, or dashboards |
| Management zone changes   | Yes     | No new zones or rule modifications                    |
| Auto-tag rule changes     | Yes     | No new tags or rule modifications                     |
| SLO changes               | Yes     | No new SLOs or metric expression changes              |
| OneAgent deployments      | No      | New agent deployments continue (they auto-register)   |
| Application deployments   | No      | Normal CI/CD continues                                |
| detected event generation | No      | Monitoring remains fully active                       |

### Freeze Communication Template

Send to all Dynatrace users and stakeholders:

SUBJECT: Dynatrace Configuration Freeze – [DATE] to [DATE]

A configuration freeze is in effect for the Dynatrace source tenant from [START DATE/TIME] to [END DATE/TIME].

During this period:

- Do NOT create or modify dashboards, SLOs, alerting rules, or settings
- Do NOT create or modify management zones or auto-tag rules
- Normal application deployments and monitoring continue as usual

Reason: SaaS-to-SaaS migration in progress. Configuration will be imported to the target tenant and changes made after the export snapshot will not be migrated.

Contact: [Migration Lead] for exceptions.

## Monitoring for Freeze Violations

Use the audit log to detect any configuration changes made during the freeze window:

```
// Source tenant: detect configuration changes during freeze window
// Run this periodically during the freeze to catch violations
// Data object corrected 09/24/2026. The Dynatrace audit trail is NOT in
`logs`: the former
// `fetch logs | filter matchesPhrase(log.source, "audit")` matched nothing,
or matched an
// unrelated file-based audit log (a database .aud file on the validation
tenant). Environment
// audit records are structured events in `dt.system.events` with event.kind
== "AUDIT_EVENT".
// Covers Settings 2.0 objects (alerting, detection rules, management zones,
auto-tags, ...).
// Token operations are a separate provider (API_TOKEN), so they are already
excluded.
// Review user.id "UNKNOWN" rows before treating them as violations: on the
validation
tenant they were metric-metadata and extension-definition writes.
fetch dt.system.events, from:-24h
| filter event.kind == "AUDIT_EVENT" and event.provider == "SETTINGS"
| filter in(event.type, {"CREATE", "UPDATE", "DELETE"})
| filter user.id != "system"
| summarize changes = count(), last_change = takeMax(timestamp), by:{user.id,
details.dt.settings.schema_id}
| sort changes desc
| limit 20
```

## 7. Rollback Procedure

Every migration must have a documented rollback procedure. Rollback is possible at any point during Steps 4–6 because the source tenant remains fully operational.

### Rollback Decision Criteria

| Signal                                                    | Severity | Rollback?                                            |
|-----------------------------------------------------------|----------|------------------------------------------------------|
| Entity count < 80% of baseline                            | Critical | Yes — agents not connecting                          |
| No log data in target after 1 hour                        | Critical | Investigate first, rollback if unresolved in 2 hours |
| detected problems in target tenant unrelated to migration | Low      | No — expected during transition                      |
| Dashboard shows no data                                   | Medium   | No — likely entity ID remapping issue (fixable)      |
| IAM users cannot log in                                   | Critical | Yes — SSO misconfiguration                           |

### Rollback Steps

| Step | Action                                                         | Time Estimate     |
|------|----------------------------------------------------------------|-------------------|
| 1    | Revert OneAgent <code>--set-server</code> to source tenant URL | 15 min per wave   |
| 2    | Revert DynaKube CR <code>apiUrl</code> to source tenant        | 5 min             |
| 3    | Verify agents reconnect to source tenant                       | 15 min            |
| 4    | Lift configuration freeze on source tenant                     | Immediate         |
| 5    | Notify stakeholders                                            | Immediate         |
| 6    | Post-mortem and re-plan                                        | Next business day |

**Key principle:** The source tenant is never decommissioned until the migration is fully validated (Step 9). Rollback is always possible by pointing agents back to the source tenant.

## 8. Step Completion Checklist

Before proceeding to Step 5 (Execute), verify all preparation tasks are complete:

| Deliverable                     | Status                   | Owner               | Notes                                                 |
|---------------------------------|--------------------------|---------------------|-------------------------------------------------------|
| Target tenant provisioned       | <input type="checkbox"/> | Platform            | Environment active, Grail enabled                     |
| SSO configured and tested       | <input type="checkbox"/> | Security / Platform | At least two group memberships validated              |
| IAM deployed via Terraform      | <input type="checkbox"/> | Platform            | Groups, policies, bindings applied                    |
| Monaco export completed         | <input type="checkbox"/> | Platform            | Full export validated with dry-run                    |
| Entity ID audit completed       | <input type="checkbox"/> | Platform            | Hardcoded IDs identified and remapping plan ready     |
| ActiveGates deployed            | <input type="checkbox"/> | Platform / Infra    | All zones covered, connectivity validated             |
| K8s operator prepared           | <input type="checkbox"/> | Platform / K8s      | DynaKube CR ready, secrets created, Helm validated    |
| Configuration freeze active     | <input type="checkbox"/> | All                 | Freeze communicated, monitoring for violations active |
| Rollback procedure documented   | <input type="checkbox"/> | Platform            | Reviewed and approved by migration lead               |
| Pre-migration baseline recorded | <input type="checkbox"/> | Platform            | Entity counts, bucket inventory captured              |

**Gate check:** Do not proceed to Step 5 until all items are checked. Missing preparation creates compounding problems during execution.

# Next Step

Continue to **S2S-05: Step 5 — Execute: Configuration Import and Agent Cutover** to import configuration and redirect agents to the target tenant.

| Completed                                                               | Next                  | Remaining                                             |
|-------------------------------------------------------------------------|-----------------------|-------------------------------------------------------|
| ~~1. Discover~~ → ~~2. Strategize~~ →<br>~~3. Design~~ → ~~4. Prepare~~ | <b>5.<br/>Execute</b> | 6. Integrate → 7. Expand → 8.<br>Enable → 9. Optimize |

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*